Chapter 2 – Servlets Part -1 - Overview of Java Web Projects and the web.xml

Contents

2.4.1 Introduction 2

2.4.1.1 Types of Java Web Projects 2

2.4.1.2 Deployment Descriptor (web.xml) 3

2.4.1.3 Elements of web.xml 3

2.4.2 Introduction - 5

2.4.2.1 Features of Servlets and its usage 5

2.4.2.2 Hierarchy of Servlets 7

..... 7

2.4.2.3 Servlet Life Cycle 9

2.4.2.4 Advantages of Servlet 12

2.4.2.5 Types of Servlets 13

2.4.2.6 Types of Servlets – Example 15

2.4.2.7 The doGet () and doPost () methods 18

2.4.2.8 Configuring Servlets using web.xml 21

2.4.2.9 Calling Servlets from form action – Example 32

2.1 Objectives

The objectives of the course are the below

- To understand the web.xml and its position in the Java web project structure
- To understand the root tag of web.xml file, its subtags and their importance
- To understand the concept of Servlets and the types of Servlet
- To understand Servlet programming and the Servlet API

2.2 Audience

This document is designed for Java EE Learners and programmers with a need to understand the basics of Java Web projects and about web.xml file. The trainees who have understood the basics of Java technology and understood the basics of Web Applications are the intended audience.

2.3 Prerequisites

Associates should have gone through all the concepts of the core skills phase, specifially the core java concepts. Web Application Basic knowledge is also a prerequisite.

2.4 Course Content

2.4.1 Introduction

Web projects hold all of the Web resources that are created and used when developing your Web application. Broadly we have two types of web project as mentioned below.

- ✓ Static Web Projects
- ✓ Dynamic Web Projects

2.4.1.1 Types of Java Web Projects

Dynamic web projects contains dynamic Java EE resources like Servlets, JSP files, filters, and associated metadata. They also contain the static resources like images and HTML files.

Static web projects contains the static resources such as images and HTML content. Static projects does not have the capability to change based on the need.

2.4.1.2 Deployment Descriptor (web.xml)

A web application's deployment descriptor describes the classes, resources and configuration of the application and how the web server uses them to serve web requests. When the web server receives a request for the application, it uses the deployment descriptor to map the URL of the request to the code that ought to handle the request.

The deployment descriptor is a file named web.xml. It resides in the app's WAR under the WEB-INF/ directory. The file is an XML file whose root element is <web-app>.

Here is a simple web.xml example that maps all URL paths (/*) to the servlet class mysite.server.ComingSoonServlet:

```
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="2.5">
   <servlet>
      <servlet-name>comingsoon</servlet-name>
      <servlet-class>mysite.server.ComingSoonServlet</servlet-class>
   </servlet>
   <servlet-mapping>
      <servlet-name>comingsoon</servlet-name>
      <url-pattern>/*</url-pattern>
   </servlet-mapping>
</web-app>
```

2.4.1.3 Elements of web.xml

The root tag in the web.xml file is the <web-app> tag. This tag can contain the elements of the deployment descriptor listed.

- ✓ context-param
- ✓ description
- ✓ display-name
- ✓ distributable
- ✓ ejb-ref
- ✓ ejb-local-ref
- ✓ env-entry
- ✓ error-page
- ✓ filter
- ✓ filter-mapping
- ✓ icon
- ✓ listener
- ✓ login-config
- ✓ mime-mapping
- ✓ resource-env-ref
- ✓ resource-ref
- ✓ security-constraint
- ✓ security-role
- ✓ servlet
- ✓ servlet-mapping
- ✓ session-config
- ✓ taglib
- ✓ welcome-file-list

Of these, the most commonly used tags will be dealt in the forthcoming sections with the details of configuration.

2.4.2 Introduction -

Servlet technology was the original Java based solution for web development. The servlet is a Java programming language class used to extend the capabilities of a server. Although servlets can respond to any types of requests, they are commonly used to extend the applications hosted by web servers, so they can be thought of as Java applets that run on servers instead of in web browsers (client machines). These kinds of servlets are the Java counterpart to other dynamic Web content technologies.

Servlets are extensively used in cases which require processing at the server side and generation of dynamic response. For example, if there is an application which takes the user credentials validates and loads dynamically based on the user's profile we would prefer servlets

2.4.2.1 Features of Servlets and its usage

Servlets are most often used to-

- Process data that was submitted from an HTML form
- Provide dynamic content to user such as the results of a database query
- Manage state information that does not exist in the stateless HTTP protocol, such as filling the articles into the shopping cart of the appropriate customer

Technically speaking, a "**servlet**" is a Java class in Java EE that conforms to the Java Servlet API, a standard for implementing Java classes which respond to requests. Servlets could in principle communicate over any client–server protocol, but they are most often used with the HTTP protocol. Thus "servlet" is often used as shorthand for "HTTP servlet". Thus, a software developer may use a servlet to add dynamic content to a web server using the Java platform. The generated content is commonly HTML, but may be other data such as XML.

To deploy and run a servlet, a web container must be used. A web container (also known as a servlet container) is essentially the component of a web server that interacts with the servlets. The web container is responsible for managing the lifecycle of servlets, mapping a URL to a particular servlet and ensuring that the URL requester has the correct access rights.

The steps involved are :

- ✓ User/Client sends a request for a Servlet
- ✓ The Web Server handles the request and communicates to the respective Servlet
- ✓ The Servlet processes the request and construct a dynamic response which is passed to the client through

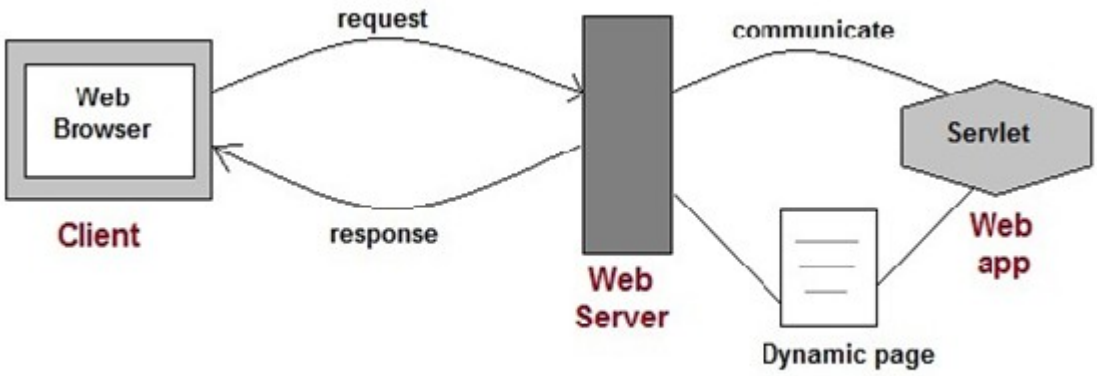

Figure1: How Servlets Work

2.4.2.2 Hierarchy of Servlets

The **Servlet interface** provides the following methods that manage the lifecycle of a servlet and its communications with clients.

- **destroy()**: Cleans up all resources that are being held by the servlet and makes sure that any persistent state is synchronized with the servlet's current in-memory state.
- **getServletConfig()**:Returns a servlet config object, which contains initialization parameters and startup configuration for this servlet.
- **getServletInfo()**: Returns a string containing information about the servlet, such as its author, version, and copyright.
- **init(ServletConfig)** : Initializes the servlet. Runs once per servlet instance. .
- **service(ServletRequest, ServletResponse)** : Carries out a single request from the client.

All servlets implement this interface, either directly or, more commonly, by extending a class that implements it such as HttpServlet. The inheritance hierarchy looks as follows:

Figure 2- Hierarchy of a servlet

Servlet writers provide some or all of these methods when developing a servlet.

When a servlet accepts a service call from a client, it receives two objects, **ServletRequest** and **ServletResponse.** The ServletRequest class encapsulates the communication from the client to

the server, while the ServletResponse class encapsulates the communication from the servlet back to the client.

A servlet container is acompiled, executable program. The function of the container is to load, initialize and execute Servlets. A container can hold large number of requests. Ingeneral, a servlet container does the following for each request for a servlet

- Loads the servlet class and calls the init method of the servlet as the servlet is called for the first time
- Makes the instance of javax.servlet.ServletRequest and javax.servlet.ServletResponse for each request
- Passes the ServletRequest and ServletResponse by invoking the servlet's service method
- Calls the destroy method and unload the servlet class when the servlet class is to be shut down

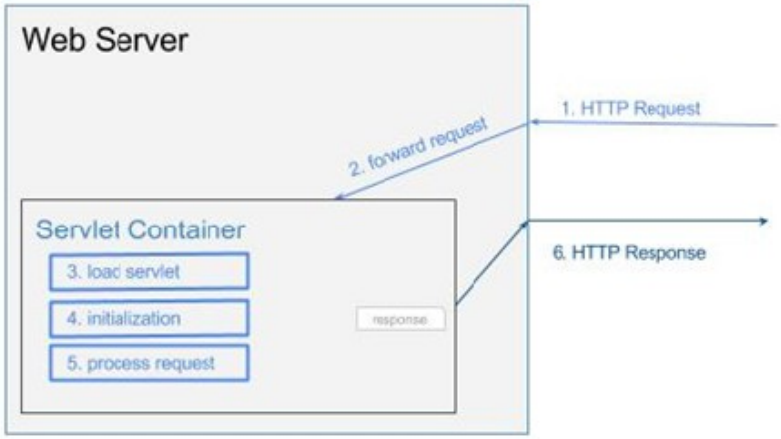

Figure 2- Role of a Servlet container

The **ServletRequest** interface defines an object to provide client request information to servlet. This interface has a set of methods which helps to retrieve the information like name and values of

parameters passed as part of client request, host name of the server, port number of the server etc. It contains every other information about the client request

The **ServletResponse** interface defines an object to assist a servlet in sending a response to the client. It has a wide set of methods to send the binary or character data back to the client.

The classes and interfaces described above make up a basic Servlet. HTTP servlets have some additional objects that provide session-tracking capabilities. The servlet writer can use these APIs to maintain state between the servlet and the client that persists across multiple connections during some time period.

2.4.2.3 Servlet Life Cycle

A servlet is basically a small Java program that runs within a Web server. It can receive requests from clients and return responses. The whole life cycle of a servlet breaks up into 3 phases:

- **Initialization**: A servlet is first loaded and initialized usually when it is requested by the clients. Some websites allow the users to load and initialize servlets whenthe server is started up so that the first request will get responded more quickly.
- **Service**: After initialization, the servlets serve clients on request, implementing the application logic of the web application they belong to.
- **Destruction**: When all pending requests are processed and the servlets have been idle for a specific amount of time, they may be destroyed by the server and release all theresources they occupy

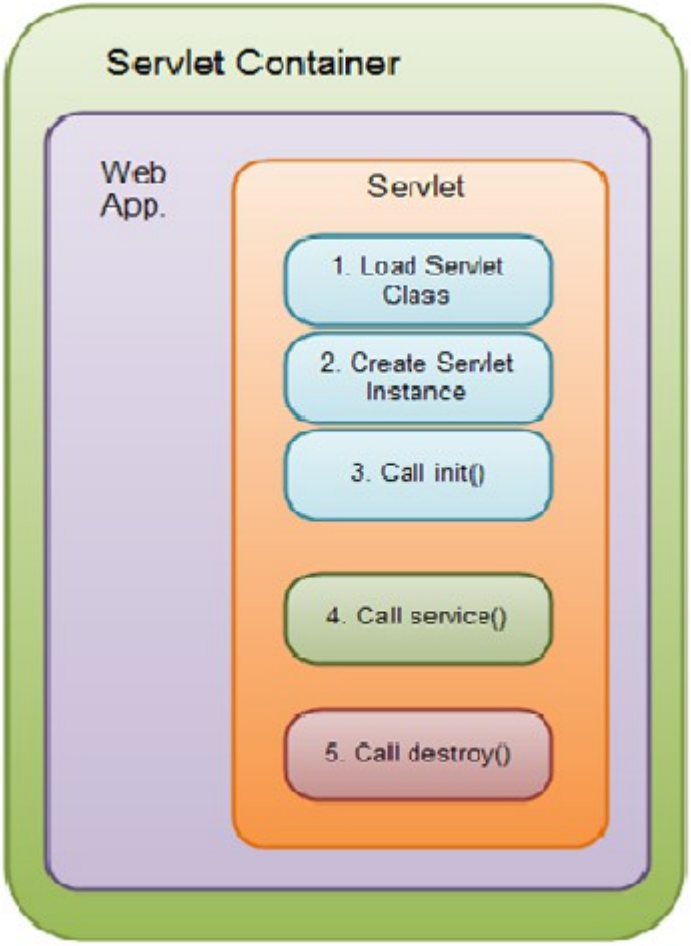

Figure 4-Life Cycle of a Servlet

After initialization, the servlet instance can service client requests. Each request is serviced in its own separate thread. The web container calls the service() method of the servlet for every

request. The service () method intern may explicitly call any other method of any java class. The developer of the servlet must provide an implementation for these methods.

Finally, the web container calls the destroy() method that takes the servlet out of service. The destroy() method, like init(), is called only once in the lifecycle of a servlet.

The following is a **typical user scenario** of these methods.

Assume that a user requests to visit a URL.

1. The browser then generates a request for this URL.
2. This request is then sent to the appropriate server.
3. The request is received by the web server and forwarded to the servlet container.
4. The container maps this request to a particular servlet.
5. The servlet is dynamically retrieved and loaded into the address space of the container.
6. The container invokes the init() method of the servlet.
7. This method is invoked only when the servlet is first loaded into memory.
8. It is possible to pass initialization parameters to the servlet so that it may configure itself.
9. The container invokes the service() method of the servlet.
10. This method is called to process the request.
11. The servlet may read data that has been provided in the request.
12. The servlet may also formulate a response for the client.
13. The servlet remains in the container's address space and is available to process any other requests received from clients.
14. The service() method is called for each request.

15. The container may, at some point, decide to unload the servlet from its memory.
16. The algorithms by which this decision is made are specific to each container.
17. The container calls the servlet's destroy() method to relinquish any resources such as file handles that are allocated for the servlet; important data may be saved to a persistent store.
18. The memory allocated for the servlet and its objects can then be garbage collected.

2.4.2.4 Advantages of Servlet

- ✓ Less response time because each request run in separate thread (as seen in the figure below)
- ✓ Servlets are scalable.
- ✓ Servlets are platform independent.
- ✓ Servlets are robust and object oriented

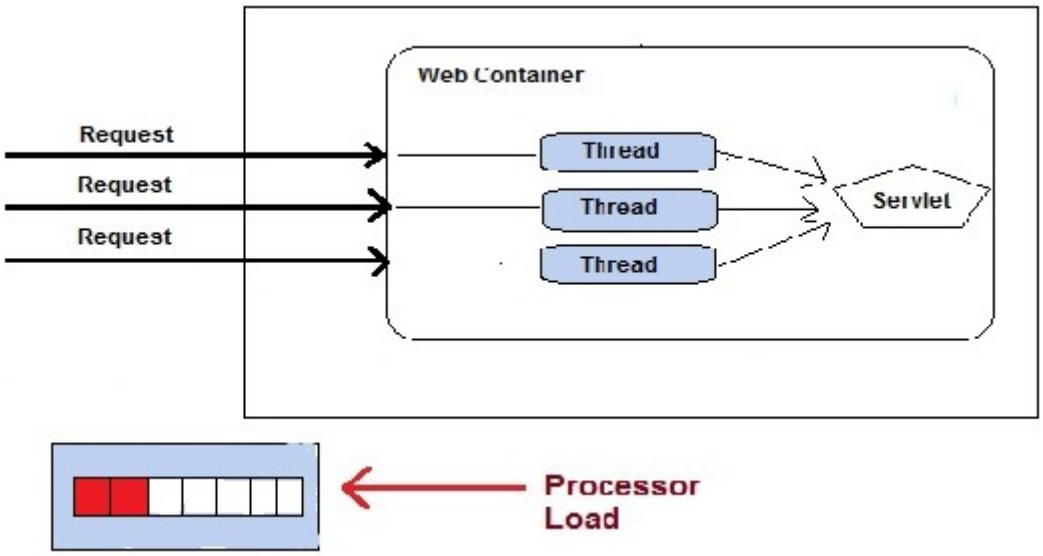

Figure 5- Servlet request creates separate thread

2.4.2.5 Types of Servlets

There are two main servlet types, **Generic and HTTP:**

- ✓ **Generic servlets**

  They extend javax.servlet.GenericServlet.

  They are protocol independent.

  They contain no inherent HTTP support or any other transport protocol.

35
17 **HTTP servlets**

They extend javax.servlet.HttpServlet.

They have built-in HTTP protocol support and are more useful in a Sun Java System Web Server environment.

They have built in methods supporting the various HTTP methods which can be called respectively based on the request.

The service method takes the HTTPServletRequest (which extends ServletRequest interface) and HTTPSerlvetResponse (which extends ServletResponse interface)interface objects as the input parameters

For both servlet types, you implement the initialization method init() and the destruction method destroy() to initialize or to release resources.

All servlets must implement a **service**() method, which is responsible for handling servlet requests. For generic servlets, simply override the service method to provide routines for handling requests. HTTP servlets provide a service method that automatically routes the request to another method in the servlet based on which HTTP transfer method is used. So, for HTTP servlets, override doPost() to process POST requests, doGet() to process GET requests, and so on.

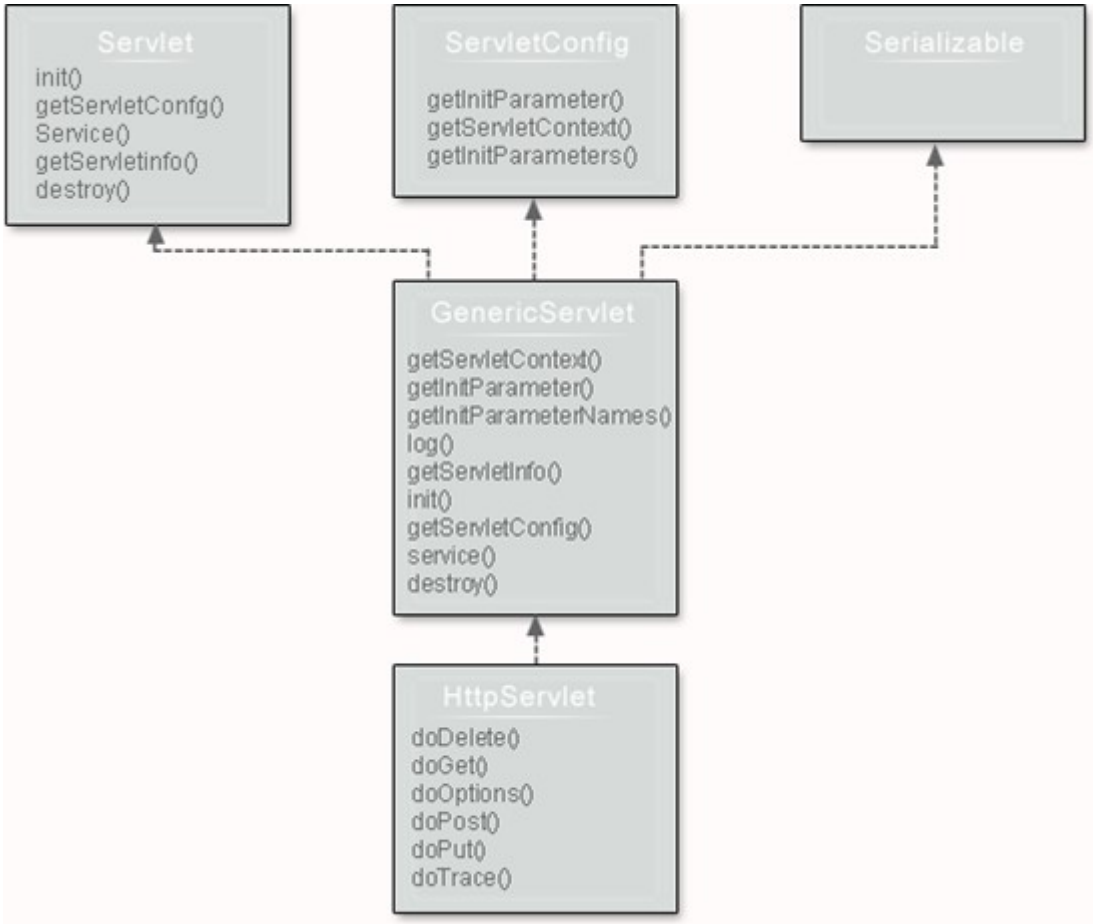

Figure 6- Servlet types and its API

2.4.2.6 Types of Servlets – Example

Generic Servlet Example:

```
iimimportjavax.servlet.GenericServlet;
iimimportjavax.servlet.ServletException;
im import javax.servlet.ServletRequest;
iimimportjavax.servlet.ServletResponse;
I    import java.io.IOException;
public pu public class ControlServlet extends GenericServlet {
public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
       // do something in here
  }
}
```

When an HTTP request arrives at the web server, targeted for your Servlet, the web server calls your Servlet's service() method.

The service() method then reads the request, and generates a response which is sent back to the client (e.g. a browser).

Here is an example service() implementation:

```
public void service(ServletRequest request, ServletResponse response)
      throws ServletException, IOException {
 String yesOrNoParam = request.getParameter("param");
```

```
  if("yes".equals(yesOrNoParam) ){
response.getWriter().write(
        "<html><body>You said yes!</body></html>");
  }
  if("no".equals(yesOrNoParam) ){
response.getWriter().write(
        "<html><body>You said no!</body></html>");
  }
}
```

This service() method first reads the request parameter "param". Then it checks if the param is equal to the text "yes" or "no", and writes an HTML response back to the browser.

HttpServlet Example:

The javax.servlet.http.HttpServlet class is a slightly more advanced base class than the GenericServlet shown in the ControlServlet example.

The HttpServlet class reads the HTTP request, and determines if the request is an HTTP GET, POST, PUT, DELETE, HEAD etc. and calls one the corresponding method.

To respond to e.g. HTTP GET requests only, you will extend the HttpServlet class, and override the doGet() method only. Here is an example:

```
public class SimpleHttpServlet extends HttpServlet {

protected void doGet( HttpServletRequest request,

HttpServletResponse response)

      throws ServletException, IOException {

response.getWriter().write("<html><body>GET response</body></html>");

  }

}
```

2.4.2.7 The doGet () and doPost () methods

The HttpServlet class has methods you can override for each HTTP method (GET, POST etc.).

Here is a list of the methods you can override:

- ✓ doGet()
- ✓ doPost()
- ✓ doHead()
- ✓ doPut()
- ✓ doDelete()
- ✓ doOptions()
- ✓ doTrace()

Most often you just want to respond to either HTTP GET or POST requests, so you just override these two methods. If you want to handle both GET and POST request from a given servlet, you can override both methods, and have one call the other.

Here is how:

```
public class SimpleHttpServlet extends HttpServlet {

        protected void doGet( HttpServletRequest request,
HttpServletResponse response)  throws ServletException, IOException {

    doPost(request, response);

  }

  protected void doPost( HttpServletRequest request,

HttpServletResponse response)

      throws ServletException, IOException {

response.getWriter().write("GET/POST response");

   }

}
```

Use the HttpServlet instead of the GenericServlet whenever possible. HttpServlet is easier to work with, and has more convenience methods than GenericServlet.

The request can be passed to a Servlet either with HTTP GET or HTTP POST methods which will call the doGet() and doPost() methods respectively.

Difference between GET and POST methods of HTTP:

GET	POST
The parameters are appended to the URL and sent along with header information.	The information is sent through a socket back to the webserver and it won't show up in the URL bar.

The amount of information you can send back using a GET is restricted as URLs can only be 1024 characters.	You can send much more information to the server this way - and it's not restricted to textual data either. It is possible to send files and even binary data such as serialized Java objects!
It is a request for information; it does not (or should not) change anything on the server.	Provides information (such as placing an order for merchandise) that the server is expected to remember
Parameters are not encrypted	Parameters are encrypted
Faster if we set the response content length since the same connection is used, thus increasing the performance	Generally used to update or post some information to the server. It is slower compared to GET since it does not write the content length
Should be idempotent. i.e. it can be repeated safely many times	Does not need to be idempotent. Operations requested through POST can have side effects for which the user can be held accountable.
Should be safe without any side effects for which user is held responsible	This method does not need to be either safe
It allows bookmarks.	It disallows bookmarks.

2.4.2.8 Configuring Servlets using web.xml

For a Java servlet to be accessible from a browser, you must tell the servlet container what servlets to deploy, and what URL's to map the servlets to. This is done in the web.xml file of your Java web application.

If you are not familiar with the directory layout and contents of a Java web application, see the second the topic: **JavaWebProjectAndWeb.xml**

Here is a list of the topics covered in the web.xml about servlet configuration:

- ✓ Configuring and Mapping a Servlet
- ✓ Servlet Init Parameters
- ✓ Servlet Load-on-Startup
- ✓ Context Parameters

To configure a servlet in the web.xml file, you write this:

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<!DOCTYPE web-app  PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"   "http://java.sun.com/dtd/web-app_2_3.dtd">

<web-app>

<servlet>

<servlet-name>controlServlet</servlet-name>

<servlet-class>com.tcs.servlet.ControlServlet</servlet-class>

</servlet>
```

```
<servlet-mapping>

<servlet-name>controlServlet</servlet-name>

<url-pattern>*.html</url-pattern>

</servlet-mapping>

</web-app>
```

Note: Sample Servlet here we have taken as com.tcs.servlet.ControlServlet.

First you configure the servlet. This is done using the **<servlet>** element.Here you give the servlet a name, and writes the class name of the servlet.

Second, you map the servlet to a URL or URL pattern. This is done in the **<servlet-mapping>**element. In the above example, all URL's ending in .html are sent to the servlet.

Other possible servlet URL mappings are:

```
/myServlet

/myServlet.do

/myServlet*
```

The * is a wild card, meaning any text. As you can see, you can either map a servlet to a single, specific URL, or to a pattern of URL's, using a wild card (*). What you will use depends on what the servlet does.

The servlet would look like

```
package com.tcs;

public class ControlServlet extends GenericServlet {
```

```
 protected String myParam = null;

        public void init(ServletConfigservletConfig) throws
ServletException{

 }

public void service(ServletRequest request, ServletResponse
response)

 }

}
```

Servlet Init Parameters:

You can pass parameters to a servlet from the web.xml file. The init parameters of a servlet can only be accessed by that servlet. Here is how you configure them in the web.xml file:

```
<servlet>

<servlet-name>controlServlet</servlet-name>

<servlet-class>com.jenkov.butterfly.ControlServlet</servlet-class>

<init-param>

<param-name>myParam</param-name>

<param-value>paramValue</param-value>

</init-param>

</servlet>
```

Here is how you read the init parameters from inside your servlet - in the servlets init() method:

```
public class ControlServlet extends GenericServlet {

  protected String myParam = null;

        public void init(ServletConfigservletConfig) throws
ServletException{

this.myParam = servletConfig.getInitParameter("myParam");

  }

public void service(ServletRequest request, ServletResponse
response)
```

```
        throws ServletException, IOException {

response.getWriter().write("<html><body>myParam = " +

this.myParam + "</body></html>");

  }

}
```

A servlets init() method is called when the servlet container loads the servlet for the first time. No one can access the servlet until the servlet has been loaded, and the init() method has been called successfully.

Servlet Load-on-Startup:

The <**servlet**> element has a subelement called **<load-on-startup>** which you can use to control when the servlet container should load the servlet. If you do not specify a <load-on-startup> element, the servlet container will typically load your servlet when the first request arrives for it.

By setting a <load-on-startup> element, you can tell the servlet container to load the servlet as soon as the servlet container starts. Remember, the servlets init() method is called when the servlet is loaded.

Here is an example <load-on-startup> configuration:

```
<servlet>

<servlet-name>controlServlet</servlet-name>

<servlet-class>com.tcs.ControlServlet</servlet-class>

<init-param><param-name>container.script.static</param-name>

<param-value>/WEB-INF/container.script</param-value>
```

```
</init-param>

<load-on-startup>1</load-on-startup>

</servlet>
```

The number inside the <load-on-startup>1</load-on-startup> element tells the servlet container in what sequence the servlets should be loaded. The lower numbers are loaded first. If the value is negative, or unspecified, the servlet container can load the servlet at any time.

Context Parameters:

You can also set some context parameters which can be read from all servlets in your application. Here is how you configure a context parameter:

```
<context-param>

<param-name>myParam</param-name>

<param-value>the value</param-value>

</context-param>
```

Here is how you access the parameter from inside an HttpServlet subclass:

```
String myContextParam =     request.getSession()

                    .getServletContext() .getInitParameter("myParam");
```

The main difference between Init parameters and Context Parameters is that Context Parameters are accessible to any servlet in the web-app whereas the Init Parameters of a servlet are specific to that servlet

The below example illustrates the difference

Consider an application which contains a Servlet which welcomes the Admin with a salutation and shows the e-mail id of the Admin and another servlet which just shows the email id.

web.xml

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<!DOCTYPE web-app  PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"  "http://java.sun.com/dtd/web-app_2_3.dtd">

<web-app>

<servlet>

<servlet-name>HelloServlet</servlet-name>

<servlet-class>com.tcs.servlet.HelloServlet</servlet-class>

<init-param>

<param-name>salute</param-name>

<param-value>Hello</param-value>

</init-param>

<init-param>

<param-name>adminName</param-name>

<param-value>John</param-value>

</init-param>

</servlet>

<servlet>
```

```
<servlet-name>MailServlet</servlet-name>

<servlet-class>com.tcs.servlet.MailServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>HelloServlet</servlet-name>

<url-pattern>*.html</url-pattern>

</servlet-mapping>

<servlet-mapping>

<servlet-name>MailServlet</servlet-name>

<url-pattern>*.do</url-pattern>

</servlet-mapping>

<context-param>

<param-name>adminEmail</param-name>

<param-value>admin@tcs.com </param-value>

</context-param>



</web-app>
```

HelloServlet.java

```
import java.io.IOException;
```

```
import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import java.io.PrintWriter;

public class HelloServlet extends HttpServlet {

 String admin = “”;

String message=””;

public  void init(ServletConfigconfig) throws ServletException {

super.init(config);

message = config.getInitParameter(“salute”);

   admin = config.getInitParameter(“adminName”);

}

protected void doGet(HttpServletRequest request,

HttpServletResponse response) throws ServletException, IOException

  {

String mailId = getServletContext().getInitParameter(“adminEmail”);

PrintWriter out = response.getWriter();

out.println (

        "<html> \n" +
```

```
        "<head> \n" +


          "<title> My first HTML  </title> \n" +

        "</head> \n" +

        "<body> \n" +

         salute+" "+admin+",Your mail id is "+mailId+

          "</font> \n" +

        "</body> \n" +

      "</html>"

    );

  }

}
```

On passing a request to the above servlet, the below message will be shown to the user

"Hello John, Your mail id is admin@tcs.com"

MailServlet.java

```
import java.io.IOException;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;
```

```
import java.io.PrintWriter;

public class MailServlet extends HttpServlet {

 public  void init(ServletConfigconfig) throws ServletException {

super.init(config);

}


protected void doGet(HttpServletRequest request,

HttpServletResponse response) throws ServletException, IOException

  {

    String mailId = getServletContext.getInitParameter("adminEmail");

PrintWriter out = response.getWriter();

out.println (

        "<html> \n" +

       "<head> \n" +

         "<title> My first HTML  </title> \n" +

       "</head> \n" +

       "<body> \n" +


        Admin mail id is "+mailId+

         "</font> \n" +
```

```
      "</body> \n" +
     "</html>"
    );
  }
}
```

On passing a request to the above servlet, the below message will be shown to the user

"Admin mail id is admin@tcs.com"

2.4.2.9 Calling Servlets from form action – Example

This section is going to show you overview about how to call a servlet from a form.

1. Create an html file

index.html

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"      pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
        "http://www.w3.org/TR/html4/loose.dtd">
<html>        <head>
                <meta      http-equiv="Content-Type"      content="text/html; charset=ISO-8859-1">
```

```
            <title> Sample HTML calling Servlet </title>
        </head>
        <body>
        <form action="HelloServlet">
                    Please enter a color <br>
                    <input type="text" name="color"size="20px">
                    <input type="submit" value="submit">
            </form>                    </body>      </html>
```

2. Create a Servlet Class

FirstServlet.java

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.PrintWriter;
public class FirstServlet extends HttpServlet {
  protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
```

```
 {
   // reading the user input
   String color= request.getParameter("color");
PrintWriter out = response.getWriter();
out.println (
        "<html> \n" +
       "<head> \n" +
         "<meta http-equiv=\"Content-Type\" content=\"text/html;
           charset=ISO-8859-1\"> \n" +
         "<title> My first HTML  </title> \n" +
       "</head> \n" +
       "<body> \n" +
         "<font size=\"12px\" color=\"" + color + "\">" +
           "Hello TCS" +
         "</font> \n" +
       "</body> \n" +
     "</html>"
   );
 }
}
```

3. Define your servlet in "web.xml"

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app id="WebApp_ID" version="2.4"

xmlns="http://java.sun.com/xml/ns/j2ee"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee

    http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">

<servlet>

<servlet-name>FirstServlet</servlet-name>

<servlet-class>FirstServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>FirstServlet</servlet-name>

<url-pattern>/FirstServlet</url-pattern>

</servlet-mapping>

</web-app>
```